SYSC 5104 — Assignment 2

Cell-DEVS Model for UAV Search Path Planning

Rachid Fahmi
Student ID: 101415381
RachidFahmi@cmail.carleton.ca

Department of Systems and Computer Engineering
Carleton University

March 2026

Contents

I Conceptual Model **4**

1 Introduction **4**

2 Problem Description **4**

3 Conceptual Model Structure **5**

3.1 Layer 0 — UAV Position Layer 7

3.2 Layer 1 — Target Probability Layer 7

4 Cell States **7**

5 Neighborhood Definition **7**

6 Behavior Rules **8**

6.1 UAV Movement Rule 8

6.2 Probability Diffusion Rule 8

7 Model Structure Sketch **9**

8 Summary of Model Parameters **10**

II Formal Cell-DEVS Specification **11**

9 Model Overview **11**

10 Coupled Cell-DEVS Model **11**

11 Cell Space and Zone Definitions **11**

12 Neighborhood Definition **12**

12.1 Spatial Moore Neighborhood 12

12.2 Layer-Specific Neighborhoods 12

13 Border Condition **12**

14 Layer 0 — UAV Position Layer **12**

14.1 Atomic Model Definition 12

14.2 State Set 13

14.3 Local Computing Function τ_0 — Three-Step Movement 14

14.4 Movement Timing 14

14.5 Transition Functions 15

15 Layer 1 — Target Probability Layer **15**
15.1 Atomic Model Definition 15
15.2 State Set 15
15.3 Local Computing Function τ_1 — Diffusion Rule 15
15.4 Transition Functions 16

16 Coupling Function Z **16**
16.1 Cross-Layer: Probability $\rightarrow$ UAV (Z_{10}) 16
16.2 Same-Layer: UAV $\rightarrow$ UAV (Z_{00}) 16
16.3 Same-Layer: Probability $\rightarrow$ Probability (Z_{11}) 16
16.4 UAV $\rightarrow$ Probability (Z_{01}) 16

17 Initial Conditions **16**
17.1 UAV Layer 16
17.2 Probability Layer 16

18 Experimental Frame **17**

19 Model Parameters **18**

III Implementation and Simulation Results **19**

20 Cadmium Implementation **19**

21 Deviations from Formal Specification **20**

22 Simulation Setup **20**

23 Simulation Results **21**
23.1 Probability Diffusion 21
23.2 UAV Path Behaviour 22
23.3 Oscillation Near the Target 23

24 Visualization with the Web Viewer **24**

25 Video Export **24**

Part I

Conceptual Model

1 Introduction

Unmanned Aerial Vehicles (UAVs) are increasingly used in surveillance, search-and-rescue, and reconnaissance missions where a target's exact location is not known in advance. In these situations, the UAV cannot simply fly directly to the target — it must navigate based on whatever partial information is available, such as sensor readings, prior intelligence, or estimates from other vehicles.

A common way to handle this kind of uncertainty is through **probabilistic occupancy maps**: two-dimensional grids where each cell stores a probability value representing the likelihood that the target is in that location. The UAV reads this map and steers toward areas with the highest probability, following the gradient of the belief field. As time passes, the map evolves — probability spreads to neighboring cells through a **diffusion** process that models the possibility that the target has moved [1].

This project builds a simplified **Cell-DEVS model** of UAV search behavior. Cell-DEVS (Cellular Discrete Event System Specification) combines cellular automata with DEVS timing, making it well-suited for spatially distributed systems that evolve through local rules [2]. The model captures two processes: the UAV moving across a grid, and the target probability field evolving through local diffusion. The design is based on the framework by Shem, Mazzuchi, and Sarkani [1], adapted into Cell-DEVS and implemented in Cadmium [3].

2 Problem Description

The problem is straightforward: a single UAV is deployed over a search area to find a target whose position is unknown. The UAV starts at a known position and must plan its path using only local information — it cannot see the full map, only its immediate neighborhood.

The uncertainty about the target's location is represented as a **probability distribution** over a discretized grid. Each cell holds a real number between 0 and 1 indicating how likely the target is to be there. This might start as a hotspot centered on the last known target location, or any other prior distribution.

Two things happen at every simulation step:

1. The UAV moves one cell, heading toward the neighbor with the highest probability — a **gradient hill-climbing strategy** [1].
2. The probability field **diffuses** outward, representing growing uncertainty about where the target may have moved. Each cell's new value is a weighted combination of itself and its neighbors [1].

The main modeling challenge is that UAV motion is a **token-movement problem** — exactly one cell represents the UAV at any time — while the probability field is a classical **field-evolution problem** suited to cellular automata. The two interact: the UAV reads the field to decide where to move, and when it visits a cell, that cell's probability can be reduced.

3 Conceptual Model Structure

The search environment is a $\mathbf{30} \times \mathbf{30} \times \mathbf{2}$ three-dimensional Cell-DEVS grid. The first two dimensions (i, j) are spatial coordinates. The third dimension k is the **layer index**, separating two components with different state types and update rules.

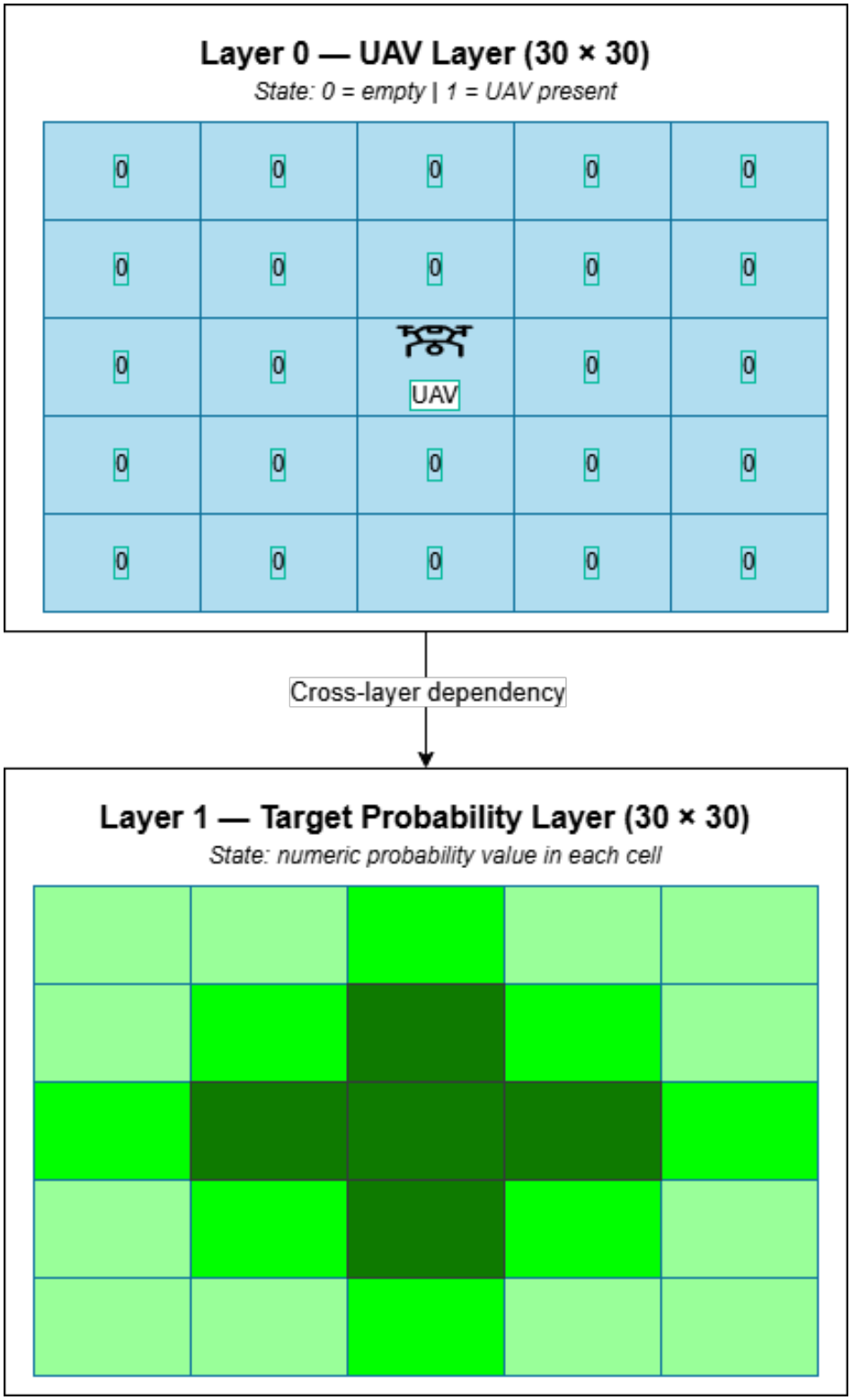

Figure 1: Two-layer Cell-DEVS model structure. Layer 0 tracks the UAV position. Layer 1 stores the target probability field. The UAV layer reads from Layer 1 to decide its next move.

3.1 Layer 0 — UAV Position Layer

Layer 0 tracks where the UAV is. Each cell is either empty or occupied. At any point in the simulation, exactly one cell holds the occupied state. This single-token constraint is what makes UAV movement non-trivial in Cell-DEVS, as discussed in Section 6.

3.2 Layer 1 — Target Probability Layer

Layer 1 stores the **target occupancy probability** (TOP) for every cell [1]. Each value is a real number between 0 and 1. This layer evolves over time through the diffusion rule (Section 6.2), reflecting growing uncertainty about where the target has moved.

4 Cell States

Table 1: Cell state definitions for each layer.

Layer	State Value	Meaning
UAV Layer (0)	`0`	Empty — no UAV present
UAV Layer (0)	`100`	UAV currently at this cell
Prob. Layer (1)	$P \in [0, 1]$	Probability the target is here

In the formal specification (Part II), the UAV layer state set is extended to $\{0, 1, \ldots, 8, 100\}$ to support a locality-safe three-step movement mechanism. Values 1–8 encode pending movement directions (N, NE, E, SE, S, SW, W, NW), ensuring exactly one cell holds state 100 at all times [3].

5 Neighborhood Definition

Both layers use a **Moore neighborhood** of radius 1, which includes the eight surrounding cells: four cardinal neighbors (N, S, E, W) and four diagonal neighbors (NE, NW, SE, SW). Formally:

$$N_M = \{(\Delta i,\, \Delta j) \mid \Delta i,\, \Delta j \in \{-1, 0, 1\}, \quad (\Delta i,\, \Delta j) \neq (0, 0)\}, \quad |N_M| = 8$$

The Moore neighborhood is used because diffusion can spread in all eight directions simultaneously, matching the motion model in [1], and the UAV gets eight movement options per step, which is more realistic than a Von Neumann neighborhood.

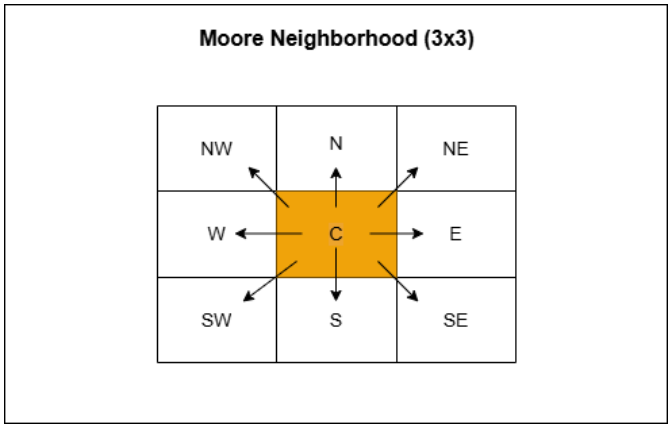

Figure 2: Moore neighborhood (3×3). The center cell interacts with its eight surrounding neighbors.

UAV-layer cells also need **cross-layer access** to the 3×3 patch of Layer 1 at the same position. This allows each UAV cell to read the probability gradient locally and decide its next move without global knowledge.

6 Behavior Rules

6.1 UAV Movement Rule

At each step, the UAV looks at the probability values of its eight neighbors and moves to the one with the highest value. Formally, if the UAV is at (i, j), the destination (i^*, j^*) is:

$$(i^*, j^*) = \arg \max_{(\Delta i, \Delta j) \in N_M} P(i + \Delta i,\ j + \Delta j,\ 1)$$

Tie-breaking: If two or more neighbors share the same maximum, the UAV picks the first in a clockwise scan starting from North:

$$\text{N} \rightarrow \text{NE} \rightarrow \text{E} \rightarrow \text{SE} \rightarrow \text{S} \rightarrow \text{SW} \rightarrow \text{W} \rightarrow \text{NW}$$

This makes results consistent and reproducible. At all times, exactly one cell holds state 100, ensuring a single UAV instance in the grid.

6.2 Probability Diffusion Rule

At each step, every cell in Layer 1 blends its current probability with its neighbors. This models growing uncertainty as the target may have moved [1]:

$$P_{i,j}^{\text{new}} = (1 - 8\alpha)\, P_{i,j} + \alpha \sum_{(\Delta i,\, \Delta j) \in N_M} P_{i+\Delta i,\, j+\Delta j}$$

where α is the **diffusion coefficient**. For the field to stay stable and non-negative:

$$\alpha < \frac{1}{8} = 0.125$$

This bound is stated in [1]: the coefficient must be less than 0.125 to guarantee stability.

An optional **visit-based reduction** can lower the probability of a visited cell by factor $(1 - \beta)$:

$$P_{i,j}^{\text{new}} = P_{i,j}^{\text{new,diff}} \cdot (1 - \beta) \quad \text{if the UAV is at } (i, j)$$

This is kept optional to keep the initial implementation simple.

7 Model Structure Sketch

The two layers interact asymmetrically:

- **Layer 1 → Layer 0:** The UAV reads probability values from Layer 1 to decide where to move.
- **Layer 0 → Layer 1 (optional):** If visit-based reduction is on, the UAV's presence lowers the probability at the visited cell.
- **Layer 1 → Layer 1:** Diffusion operates entirely within Layer 1 using Moore neighbors.

The simulation space is $30 \times 30 \times 2$. This size is large enough for meaningful dynamics while remaining fast to simulate in Cadmium.

8 Summary of Model Parameters

Table 2: Model parameters and default values.

Parameter	**Symbol**	**Valid Range**	**Default**
Grid dimensions	—	fixed	30×30
Number of layers	—	fixed	2
Neighborhood type	—	fixed	Moore, radius 1
Diffusion coefficient	α	$(0,\ 1/8)$	0.05
Visit reduction	β	$[0, 1]$	0.5
Initial UAV position	(i_0, j_0)	any cell	$(0, 0)$

Part II

Formal Cell-DEVS Specification

9 Model Overview

The UAV search system is specified as a three-dimensional cell space of dimensions $30 \times 30 \times 2$. The first two dimensions are the spatial grid; the third is the layer index. Layer 0 is the UAV position layer and Layer 1 is the target probability layer. This separation lets each layer have its own state set and local rule, while they remain coupled through the translation function Z.

10 Coupled Cell-DEVS Model

$$\text{GCC} = \langle Xlist,\ Ylist,\ I,\ X,\ Y,\ n,\ \{t_1, t_2, t_3\},\ \eta,\ N,\ C,\ B,\ Z,\ \text{select}\rangle$$

Table 3: GCC coupled model components.

Component	**Value**	**Explanation**
n	3	Three-dimensional cell space
$\{t_1, t_2, t_3\}$	$\{30, 30, 2\}$	30×30 grid, 2 layers
η	17	Max neighborhood size (η_0=17, η_1=8)
C	$\{C(i, j, l)\}$	All cells, $i, j \in [1, 30]$, $l \in \{0, 1\}$
B	non-wrapped	Absorbing border — Section 13
Z	see Section 16	Cross-layer and same-layer coupling
select	lex. on (l, i, j)	Layer 0 processed before Layer 1

11 Cell Space and Zone Definitions

Each cell is identified as $C(i, j, l)$ where i is the row, j the column, and l the layer. The cell space splits into two zones:

$$C^{UAV} = \{C(i, j, 0) \mid i, j \in [1, 30]\} \qquad \text{(Zone 0 — UAV layer)}$$

$$C^{Prob} = \{C(i, j, 1) \mid i, j \in [1, 30]\} \qquad \text{(Zone 1 — Probability layer)}$$

12 Neighborhood Definition

12.1 Spatial Moore Neighborhood

$$N_{xy} = \{(\Delta i, \Delta j) \mid \Delta i, \Delta j \in \{-1, 0, 1\},\ (\Delta i, \Delta j) \neq (0, 0)\}, \quad |N_{xy}| = 8$$

12.2 Layer-Specific Neighborhoods

Probability layer cells use same-layer neighbors only:

$$N^{Prob} = N_{xy}, \quad \eta_1 = 8$$

UAV layer cells use same-layer neighbors plus a cross-layer read of the full 3×3 patch of Layer 1:

$$N^{UAV} = N_{xy} \ \cup \ \{(\Delta i, \Delta j, +1) \mid \Delta i, \Delta j \in \{-1, 0, 1\}\}, \quad \eta_0 = 8 + 9 = 17$$

This cross-layer component lets the UAV cell read the local probability gradient and pick a movement direction, using only its 8-cell Moore neighborhood.

13 Border Condition

Non-wrapped (absorbing) borders are used. Cells outside the grid are treated as quiescent and contribute their default value to neighbor computations:

- UAV layer border: **0** (no UAV)
- Probability layer border: **0** (no probability mass)

This models a bounded search area where the target cannot be outside the grid.

14 Layer 0 — UAV Position Layer

14.1 Atomic Model Definition

$$\mathrm{TDC}_0(i, j) = \langle X_0,\ Y_0,\ S_0,\ \eta_0,\ I_0,\ N_0,\ d_0,\ \tau_0,\ \delta_0^{int},\ \delta_0^{ext},\ \lambda_0,\ D_0 \rangle$$

Table 4: TDC_0 parameters.

Parameter	**Value**
η_0	17
d_0	Transport delay
D_0	1 simulation step
X_0, Y_0	S_0

14.2 State Set

$$S_0 = \{0,\ 1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7,\ 8,\ 100\}$$

Table 5: UAV layer state encoding and direction mapping.

Code	**Meaning**	**Offset** $(\Delta i, \Delta j)$	**Step 2: neighbor that points here**
0	Empty cell	—	—
100	UAV present	—	—
1	Move North	$(-1,\ 0)$	South neighbor $(+1,\ 0)$ holds code 1
2	Move NE	$(-1, +1)$	SW neighbor $(+1, -1)$ holds code 2
3	Move East	$(0, +1)$	West neighbor $(0, -1)$ holds code 3
4	Move SE	$(+1, +1)$	NW neighbor $(-1, -1)$ holds code 4
5	Move South	$(+1,\ 0)$	North neighbor $(-1,\ 0)$ holds code 5
6	Move SW	$(+1, -1)$	NE neighbor $(-1, +1)$ holds code 6
7	Move West	$(0, -1)$	East neighbor $(0, +1)$ holds code 7
8	Move NW	$(-1, -1)$	SE neighbor $(+1, +1)$ holds code 8

14.3 Local Computing Function τ_0 — Three-Step Movement

Let $P(\Delta i, \Delta j)$ denote the Layer 1 probability at relative position $(\Delta i, \Delta j, +1)$. Rules are evaluated in order; the first match fires.

Step 1 — Decide *(fires when current cell holds the UAV)*

For direction codes $d = 1, 2, \ldots, 8$ evaluated in order:

$$\tau_0 = d \quad \text{if } (0,0,0) = 100 \;\; \text{AND} \;\; P(\delta(d)) \geq P(\delta(k)) \;\; \forall\, k \in \{1, \ldots, 8\}$$

Because rules are checked in order $d = 1 \to 8$, the first maximally probable direction fires, giving the clockwise tie-break **N** → **NE** → **E** → **SE** → **S** → **SW** → **W** → **NW** automatically.

Step 2 — Arrive *(fires when cell is empty and a neighbor points here)*

$$\tau_0 = 100 \quad \text{if } (0,0,0) = 0 \;\; \text{AND}$$
$$\big[(+1,\ 0,0) = 1 \;\; \text{OR} \;\; (+1,-1,0) = 2 \;\; \text{OR} \;\; (0,-1,0) = 3 \;\; \text{OR} \;\; (-1,-1,0) = 4$$
$$\text{OR} \;\; (-1,\ 0,0) = 5 \;\; \text{OR} \;\; (-1,+1,0) = 6 \;\; \text{OR} \;\; (0,+1,0) = 7 \;\; \text{OR} \;\; (+1,+1,0) = 8\big]$$

Step 3 — Clear *(fires when cell holds a stale direction code)*

$$\tau_0 = 0 \quad \text{if } 1 \leq (0,0,0) \leq 8$$

Default: $\tau_0 = (0,0,0)$ (no change)

14.4 Movement Timing

Full movement completes in two simulation ticks:

Table 6: Two-tick UAV movement sequence.

Tick	**Event**
$t \to t+1$	UAV cell (state = 100) fires Step 1 and emits direction code d.
$t+1 \to t+2$	Destination cell fires Step 2 (becomes 100). Old UAV cell fires Step 3 (becomes 0). Both happen in the same tick.

Exactly one cell holds state 100 at all times. Duplication is impossible because Step 2 only fires on empty cells and Step 3 clears the source in the same tick.

14.5 Transition Functions

$$\delta_0^{ext}(q, e, x): \text{ compute } \tau_0 \text{ from neighborhood; enqueue result with delay } D_0 \text{ if } \tau_0 \neq s$$

$$\delta_0^{int}(q): \text{ commit head of output queue; passivate if empty}$$

$$\lambda_0(q) = \text{head of output queue} \qquad ta_0(q) = D_0 \text{ if non-empty; } \infty \text{ otherwise}$$

Transport delay is used because once a direction is committed, the move must complete. Inertial delay could cancel a move before the UAV arrives.

15 Layer 1 — Target Probability Layer

15.1 Atomic Model Definition

$$\mathrm{TDC}_1(i, j) = \langle X_1,\ Y_1,\ S_1,\ \eta_1,\ I_1,\ N_1,\ d_1,\ \tau_1,\ \delta_1^{int},\ \delta_1^{ext},\ \lambda_1,\ D_1 \rangle$$

Table 7: TDC_1 parameters.

Parameter	Value
η_1	8
d_1	Transport delay
D_1	1 simulation step

15.2 State Set

$$S_1 = [0, 1] \subset \mathbb{R}$$

15.3 Local Computing Function τ_1 — Diffusion Rule

$$P_{i,j}^{new} = (1 - 8\alpha)\, P_{i,j} \ + \ \alpha \sum_{(\Delta i, \Delta j) \in N_{xy}} P_{i+\Delta i,\, j+\Delta j}$$

where α must satisfy $0 < \alpha < \frac{1}{8}$ to keep the self-weight positive and the field non-oscillating [1].

Clipping is applied after every update:

$$P_{i,j}^{new} = \max\bigl(0,\ \min(1,\ P_{i,j}^{new})\bigr)$$

Visit-based belief reduction is not implemented. The probability layer evolves through diffusion only.

15.4 Transition Functions

$$\delta_1^{ext}(q, e, x): \text{ compute } \tau_1 \text{ from Moore neighborhood; enqueue if changed}$$

$$\delta_1^{int}(q): \text{ commit queued value; passivate if empty} \qquad ta_1(q) = D_1 \text{ if non-empty; } \infty \text{ otherwise}$$

16 Coupling Function Z

16.1 Cross-Layer: Probability → UAV (Z_{10})

$$Z_{10} : P^Y_{(i,j,1)} \;\rightarrow\; P^X_{(i+r,\, j+s,\, 0)} \quad \text{for all } (r, s) \in \{-1, 0, 1\}^2$$

Each probability cell broadcasts its value to the UAV cell at the same position and to the eight surrounding UAV cells, enabling local gradient computation in Step 1.

16.2 Same-Layer: UAV → UAV (Z_{00})

$$Z_{00} : P^Y_{(i,j,0)} \;\rightarrow\; P^X_{(i+r,\, j+s,\, 0)} \quad \text{for all } (r, s) \in N_{xy}$$

UAV cells broadcast their state to Moore neighbors, enabling Step 2.

16.3 Same-Layer: Probability → Probability (Z_{11})

$$Z_{11} : P^Y_{(i,j,1)} \;\rightarrow\; P^X_{(i+r,\, j+s,\, 1)} \quad \text{for all } (r, s) \in N_{xy}$$

16.4 UAV → Probability (Z_{01})

Z_{01} is **not defined**. The probability layer evolves through diffusion only; visit-based reduction is out of scope for this model.

17 Initial Conditions

17.1 UAV Layer

$$U(i_0, j_0, 0) = 100, \qquad U(i, j, 0) = 0 \;\; \forall (i, j) \neq (i_0, j_0)$$

17.2 Probability Layer

$$P(i, j, 1) = P_0(i, j) \quad \forall\, i, j \in [1, 30]$$

where P_0 is the initial probability map defined in the scenario JSON configuration, with $P_0(i, j) \in [0, 1]$ for all cells.

18 Experimental Frame

Three unit-level tests are run before full integration:

- **Experiment A — Probability layer only:** Freeze the UAV layer; initialize one hotspot; verify outward diffusion and α stability.
- **Experiment B — UAV layer only:** Use a static probability map; verify hill-climbing and no UAV duplication.
- **Experiment C — Full integration:** Both layers active; UAV pursues gradient while the field evolves.

Table 8: Experimental frame scenarios.

Scenario	**UAV Init**	**Prob. Init**	**Expected Behavior**	**Measurement**
S1 — Single hotspot	$(0, 0)$	Peak at $(22, 22)$	UAV follows gradient toward peak	Steps to reach peak
S2 — Two equal peaks	$(15, 0)$	Equal peaks at $(5, 5)$ and $(5, 25)$	UAV converges on nearer peak	Which peak reached
S3 — Uniform map	$(15, 15)$	$P_0 = 0.5$ everywhere	Code 1 (N) fires every step	Straight vertical path
S4 — Diffusing hotspot	$(0, 0)$	Hotspot at $(22, 22)$, α=0.05	UAV tracks gradient as field flattens	Convergence time
S5 — α sensitivity	$(0, 0)$	Hotspot at $(22, 22)$; vary α	Small α: sharp gradient. Large α: flat gradient	Steps to peak vs α

19 Model Parameters

Table 9: Formal model parameters.

Parameter	**Symbol**	**Valid Range**	**Default**
Diffusion coefficient	α	$(0,\ 1/8)$	0.05
Grid dimensions	—	fixed	30×30
Transport delay	D	1	1
Initial UAV position	(i_0, j_0)	any cell	$(0, 0)$

Part III

Implementation and Simulation Results

20 Cadmium Implementation

The model was built in Cadmium v2 using the grid Cell-DEVS library. Each grid cell stores two values: the probability that the target is there, and a flag showing whether the UAV is currently at that cell. In code this is a small C++ struct with two fields:

```
struct UAVSearchState {
    double prob;   // target probability in [0, 1]
    int    uav;    // 0 = empty, 100 = UAV present
};
```

The formal specification in Part II describes two separate layers with explicit cross-layer coupling (Z_{10}). In the Cadmium implementation, these two layers are merged into one combined cell state. Every cell carries both the probability and the UAV flag, so the local function can read both directly from the same neighborhood without needing separate layers or inter-layer message passing. The observable behavior is the same as the two-layer specification.

The cell behavior is split across three source files:

Table 10: Source files and their roles.

File	Role
`uav_search_state.hpp`	Defines the state struct, `operator!=`, `operator<<`, and `from_json`.
`uav_search_cell.hpp`	Implements `localComputation`: the diffusion rule updates `prob`; the gradient rule updates `uav`.
`main.cpp`	Registers the `uav_search` cell type, loads the configuration, runs the simulation.

The state is printed as `prob,uav` with no spaces:

```
os << s.prob << "," << s.uav;
```

This produces CSV rows like `0;436;(0,0);;0,100`, which is exactly the format the Cell-DEVS Web Viewer expects.

21 Deviations from Formal Specification

Table 11: Differences between formal specification and implementation.

Aspect	**Specification**	**Implementation**
Layer structure	Two separate layers with cross-layer coupling Z_{10}	Single combined cell state; no explicit coupling needed
UAV state encoding	10-value set $\{0, 1..8, 100\}$ with direction codes	Binary: 0 or 100; movement decided directly in `localComputation`
Visit-based reduction	Optional β parameter (Z_{01})	Not implemented; diffusion only

22 Simulation Setup

All results in this report use the following configuration:

- Grid size: 30×30 cells.
- Neighborhood: Moore, radius 1 (8 neighbors).
- Boundary condition: non-wrapped (finite search area).
- Initial UAV position: cell $(0, 0)$, upper-left corner.
- Initial hotspot: cell $(22, 22)$ with `prob` $= 1.0$.
- Diffusion coefficient: $\alpha = 0.05$.
- Simulation length: 200 steps.

The configuration is stored in `uav_config.json`, which also includes a `viewer` section that maps the `prob` and `uav` fields to color layers in the web viewer.

23 Simulation Results

23.1 Probability Diffusion

At time step 0, nearly all cells have `prob` $= 0$. Only the hotspot at $(22, 22)$ starts at 1.0. As the simulation runs, the diffusion rule spreads probability outward in all eight directions. Cells near the hotspot gain value while the center cell gradually loses some of its own. After enough steps, a ring of medium-probability cells forms around the hotspot and keeps growing outward.

Figure 3: Initial probability map at $t = 0$. Only the hotspot cell at $(22, 22)$ has a nonzero value. The rest of the grid starts at zero.

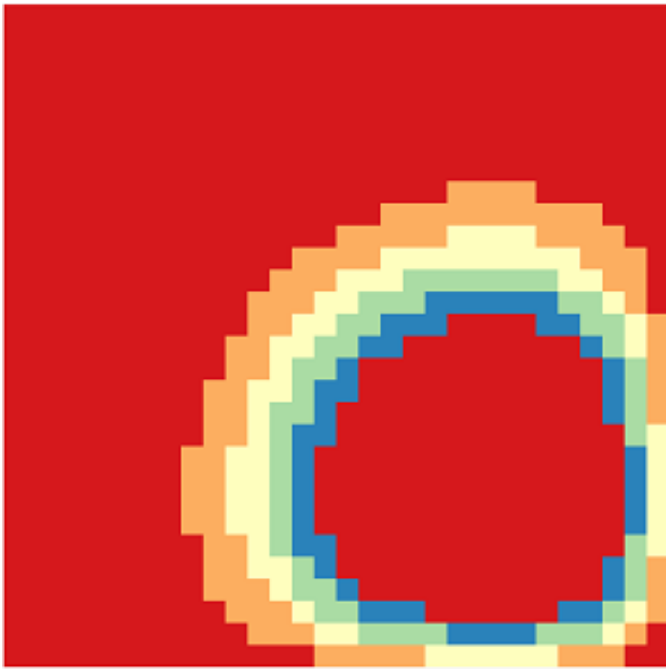

Figure 4: Probability diffusion partway through the simulation. The hotspot has spread outward into a ring shape. The center has lost some probability while neighboring cells have gained it.

23.2 UAV Path Behaviour

The UAV starts at $(0, 0)$ and at each step moves to the neighbor with the highest probability. When the field around the UAV is flat, it falls back on the clockwise tie-breaking rule and follows a predictable path. Once the diffusion wave reaches the UAV's local neighborhood, the gradient becomes detectable and the path bends toward the hotspot.

Figure 5 shows the UAV partway through the run, after it has moved away from the corner toward the high-probability region.

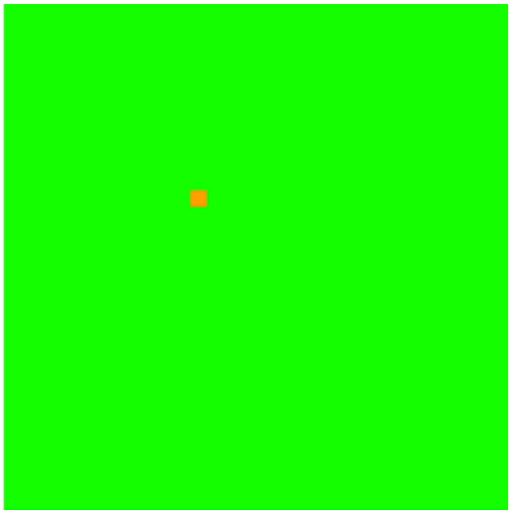

Figure 5: UAV position partway through the simulation. The single highlighted cell shows where the UAV currently is, well away from its starting corner and moving toward the hotspot.

23.3 Oscillation Near the Target

Once the UAV reaches the high-probability region around the hotspot, the probability field in that area is nearly flat because diffusion has smoothed it out. Several neighboring cells can have almost the same value, so the UAV has no clear direction to go. It ends up jumping back and forth between a small group of cells near the peak rather than staying in one place.

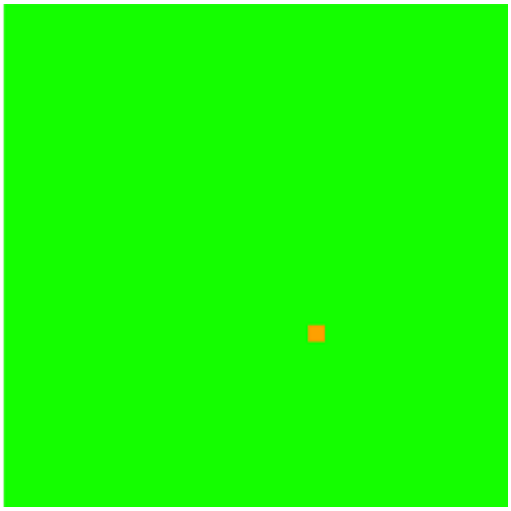

Figure 6: UAV oscillating near the target region. The local gradient is nearly flat at this point, so the UAV alternates between neighboring cells.

This is a known limitation of local gradient hill-climbing without memory. The UAV has no record of where it has been, and the probability field keeps changing due to diffusion. Adding visit-based probability reduction (the β parameter from Section 6.2) would lower

the probability at visited cells and help the UAV move on, but this was left out of the current implementation to keep things simple.

24 Visualization with the Web Viewer

All results were viewed using the Cell-DEVS Web Viewer. The raw log file `uav_log.csv` and the configuration file `uav_config.json` were uploaded directly without any post-processing. The viewer was configured with two layers: a probability layer showing the diffusion pattern, and a UAV layer highlighting the single cell where the UAV is.

Getting the viewer to work required a few things to be exactly right:

- The configuration filename had to follow the expected naming convention (`uav_config.json`).
- The field names in the `viewer` section of the JSON (`prob` and `uav`) had to match the state keys exactly.
- The `operator<<` had to print `prob,uav` with no spaces.
- The raw CSV log had to be uploaded as-is, without filtering out any rows.

Figure 7 shows both panels open at the same time.

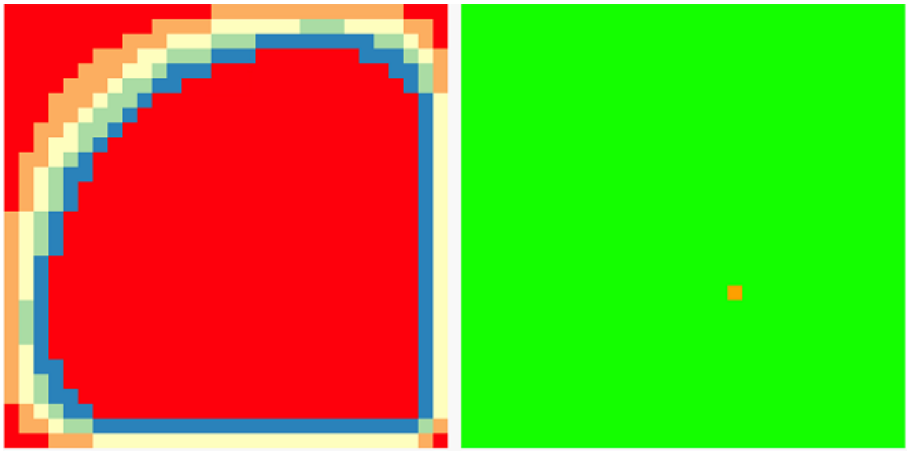

Figure 7: Cell-DEVS Web Viewer with both layers open. The left panel shows the probability diffusion pattern. The right panel shows the UAV position.

25 Video Export

A short video of the simulation was recorded using the Cell-DEVS Web Viewer and is included in the repository under `videos/`.

References

[1] A. G. Shem, T. A. Mazzuchi, and S. Sarkani, "Addressing Uncertainty in UAV Navigation Decision-Making," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 44, no. 1, pp. 295–313, Jan. 2008.

[2] G. A. Wainer, *Discrete-Event Modeling and Simulation: A Practitioner's Approach*. CRC Press, 2009.

[3] K. Holman and J. Kuzub, "UAV Path Planner with Probabilistic Terrain," SYSC 5104 Assignment 2 Report, Carleton University, Nov. 2008.